Module 14 - chapter 13

Website Monitoring 1: Scheduled Task to Monitor Application Health

The installation guide for running Docker on Debian can be found here:

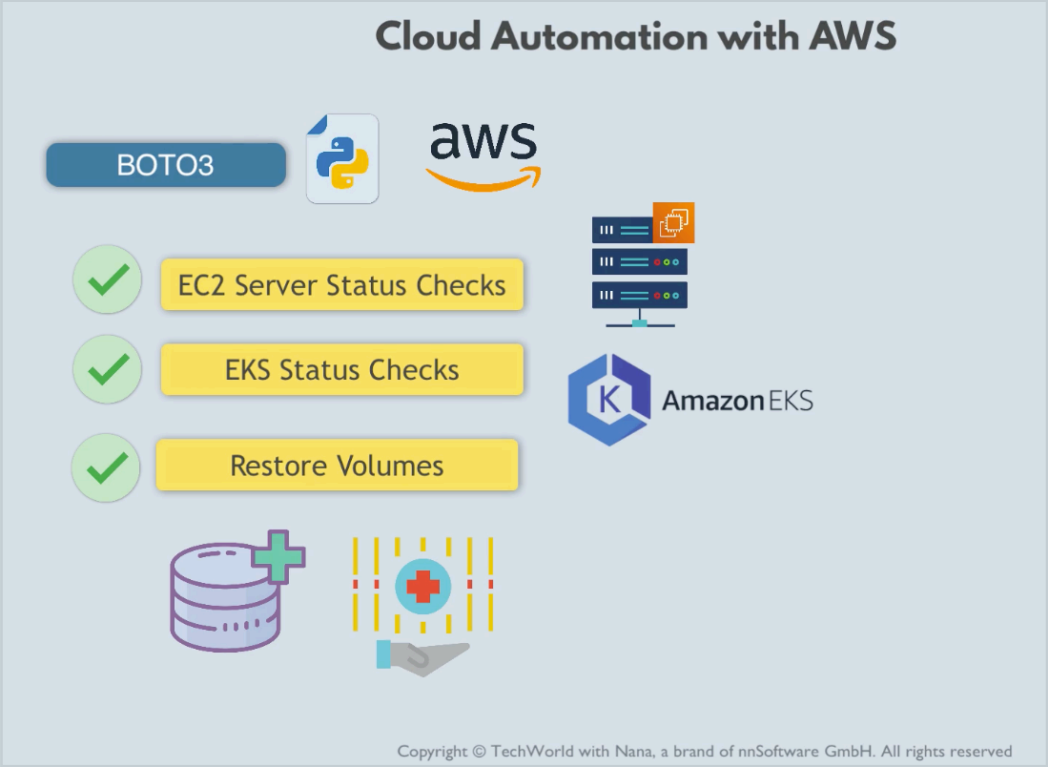
- https://docs.docker.com/engine/install/debian/

The project file used in this and the following lectures can be found here:

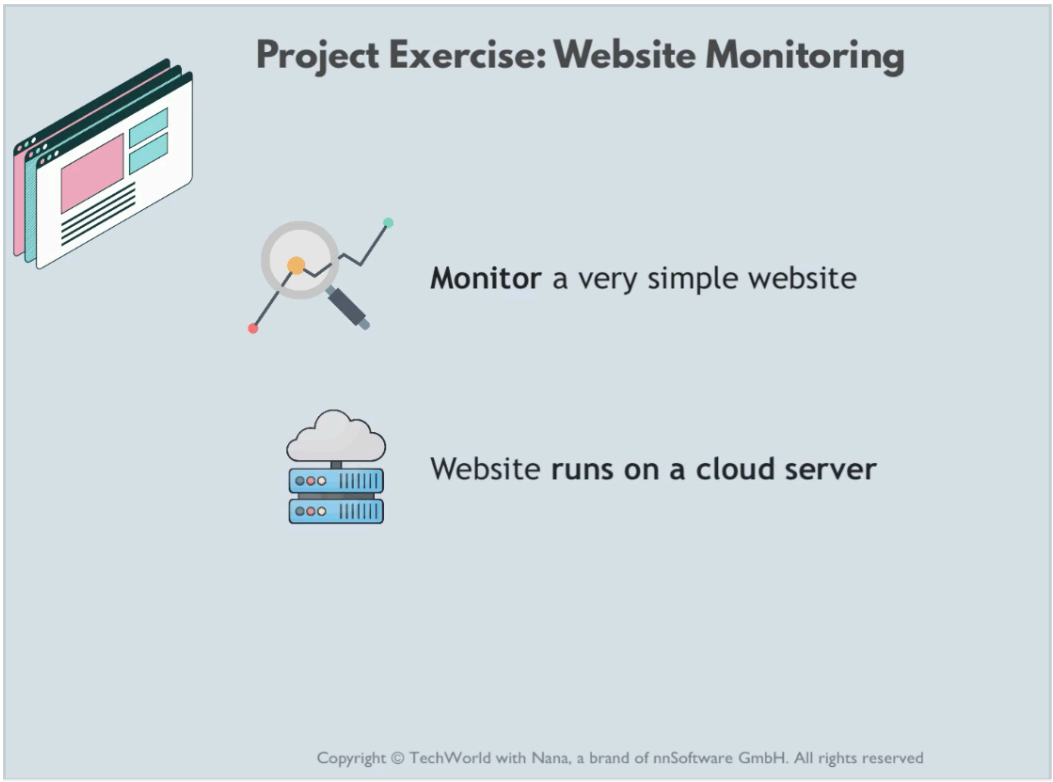
- https://gitlab.com/twn-devops-bootcamp/latest/14-automation-with-python/automation-projects/-/blob/main/monitor-website.py

Project intro

So far we have don't:



Now we will work on the USE case of Website Monitoring which is very common and is not related to AWS



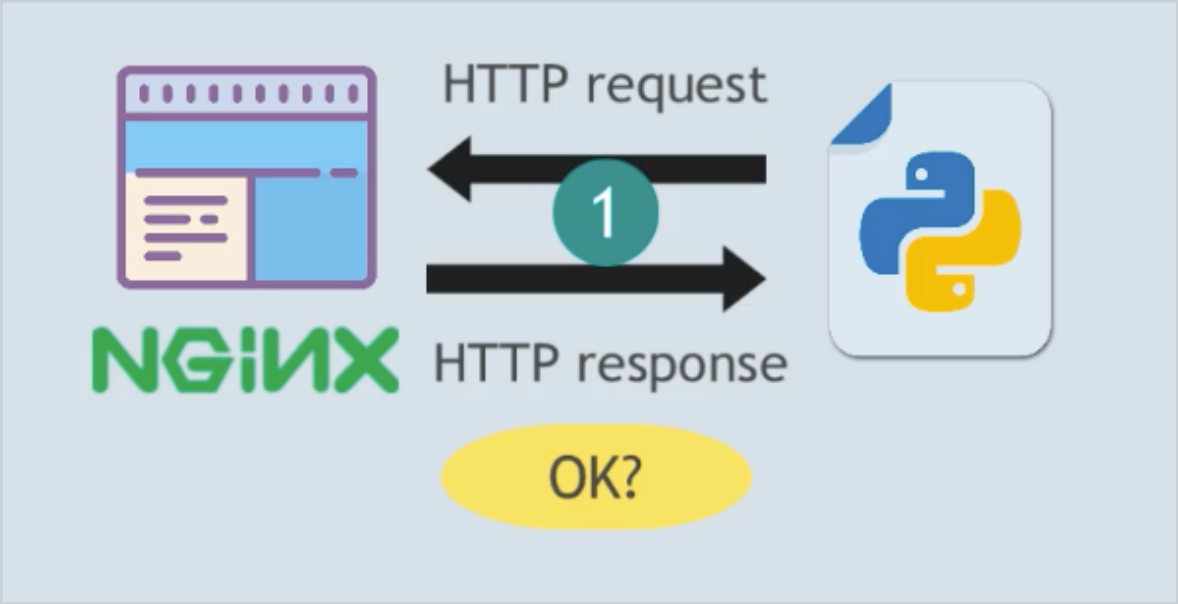
Preparations steps:

1. Create a server on Linode
2. Install docker on the server
3. Run nginx container

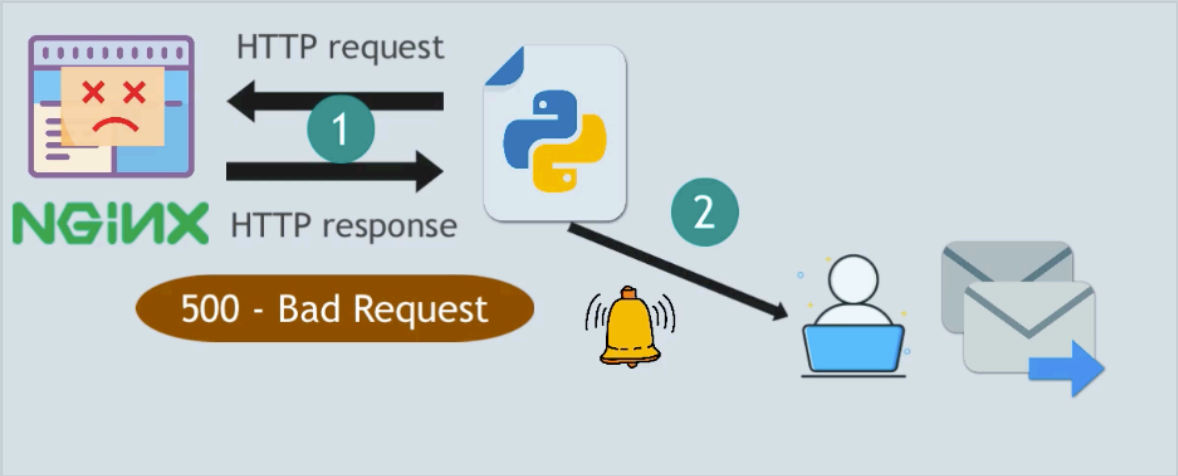
Once we have nginx up and running we can see the default welcome page on the browser (see notes Nginx Q&A)

And now we can create our Automation program to monitor the website.

The python program will check the application -> nginx container



And if something goes wrong it notifies “support” via email



1) Write Python program that **checks application**

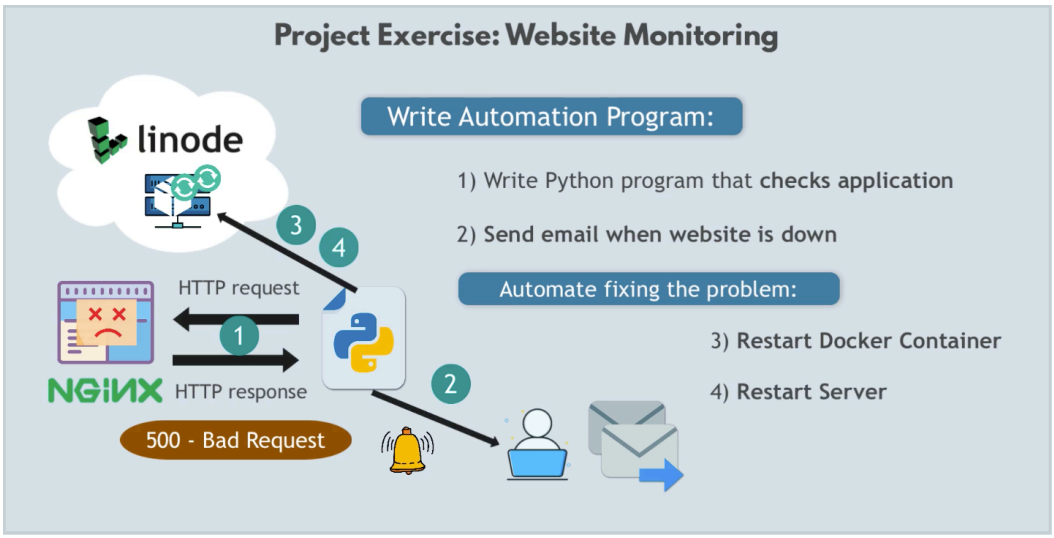
2) **Send email when website is down**

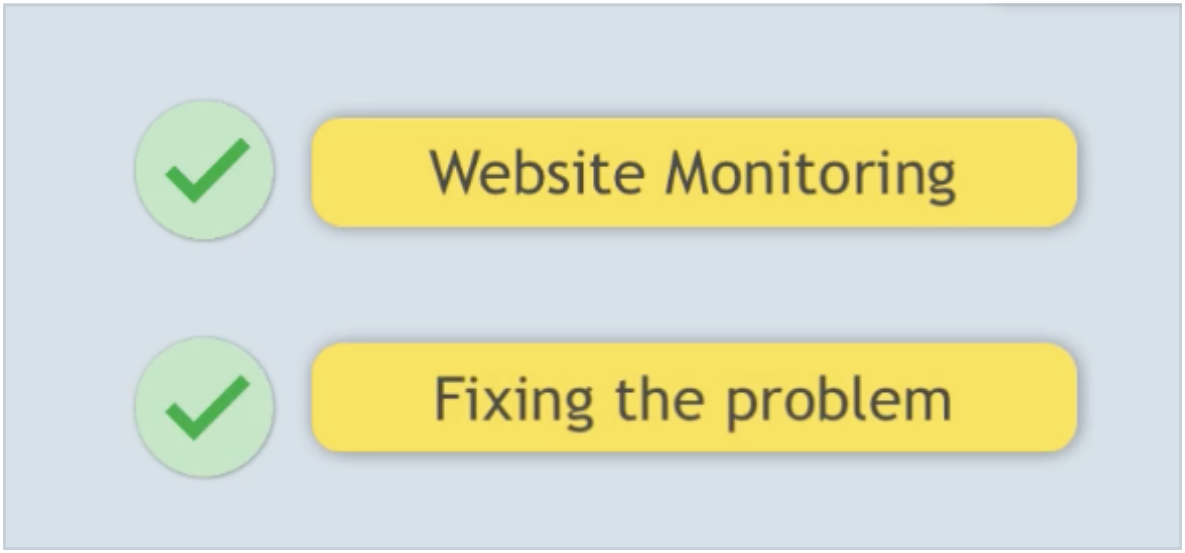
Then we will automate:

- Fixing the problem

After the notification, we will extend the python logic to restart the docker container

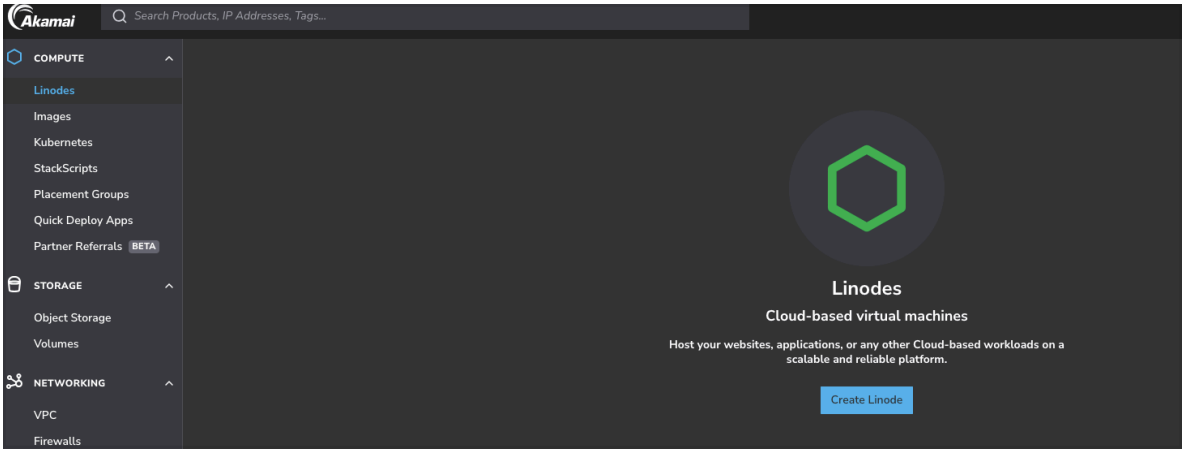
Or if the server is not accessible then -> restart the server

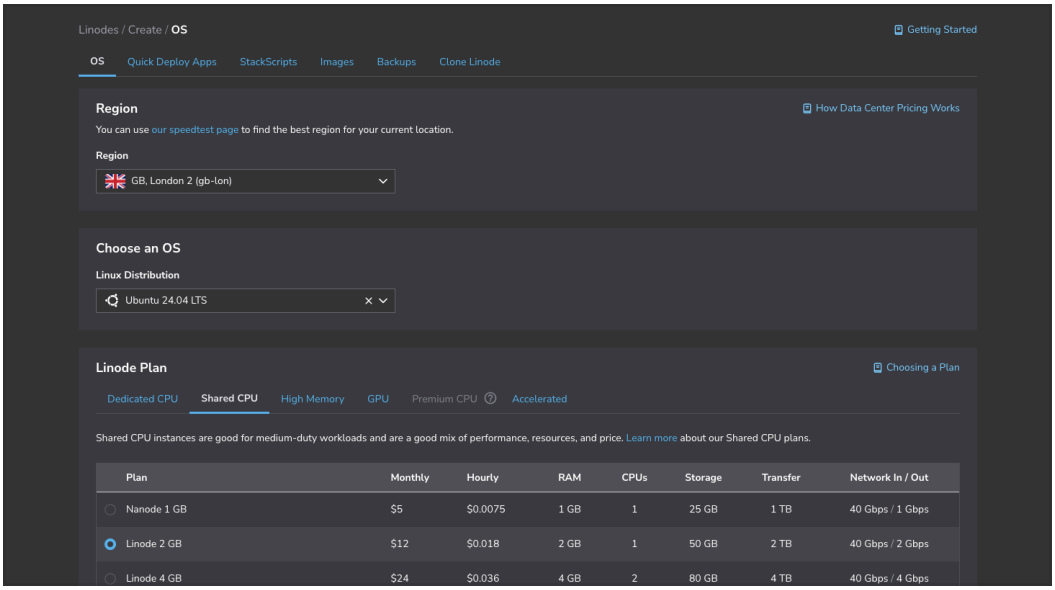




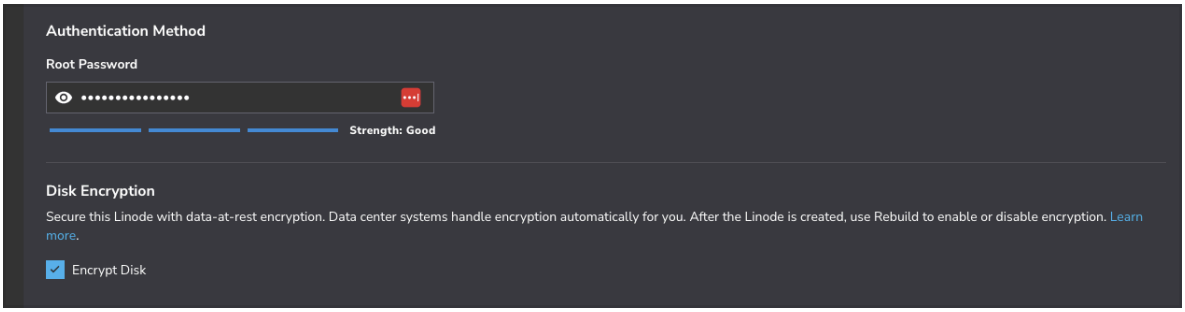
Preparation

Create server and run nginx container:

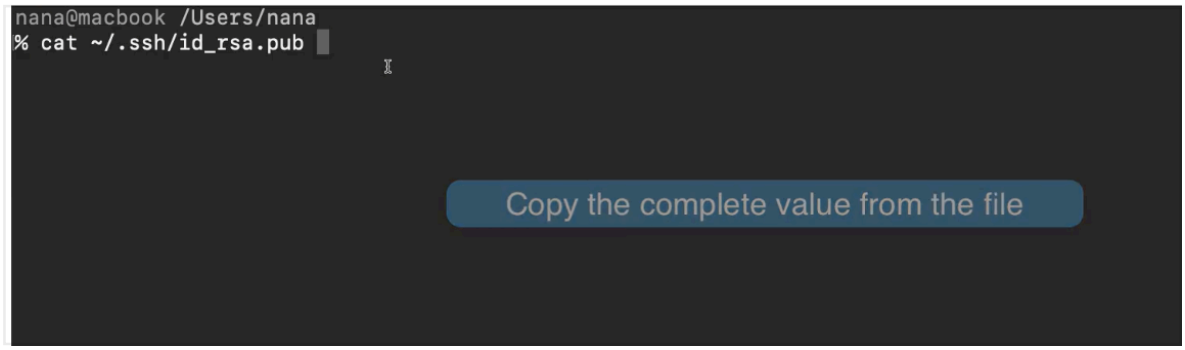




Add root password

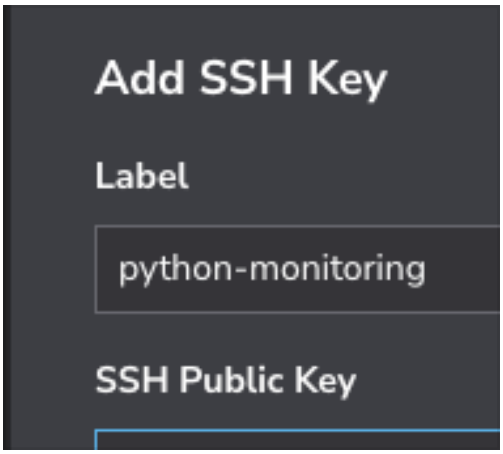


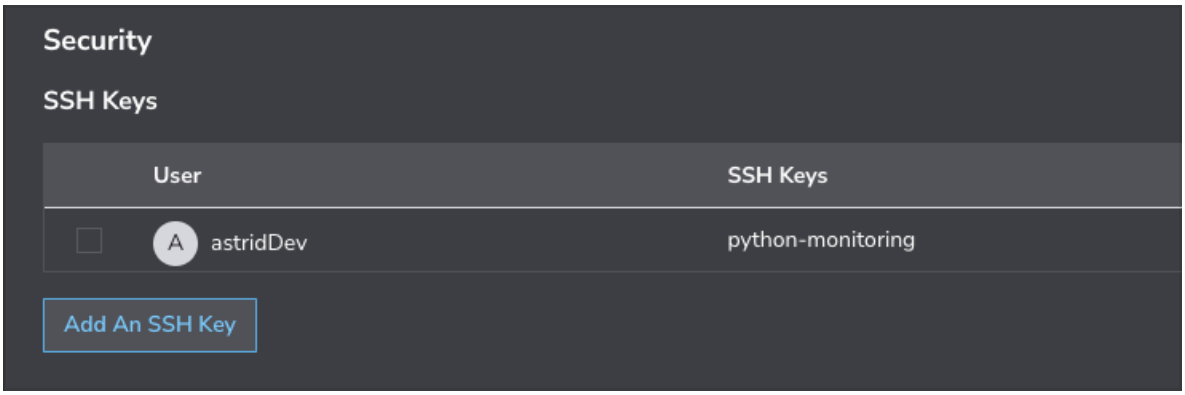
Add ssh key



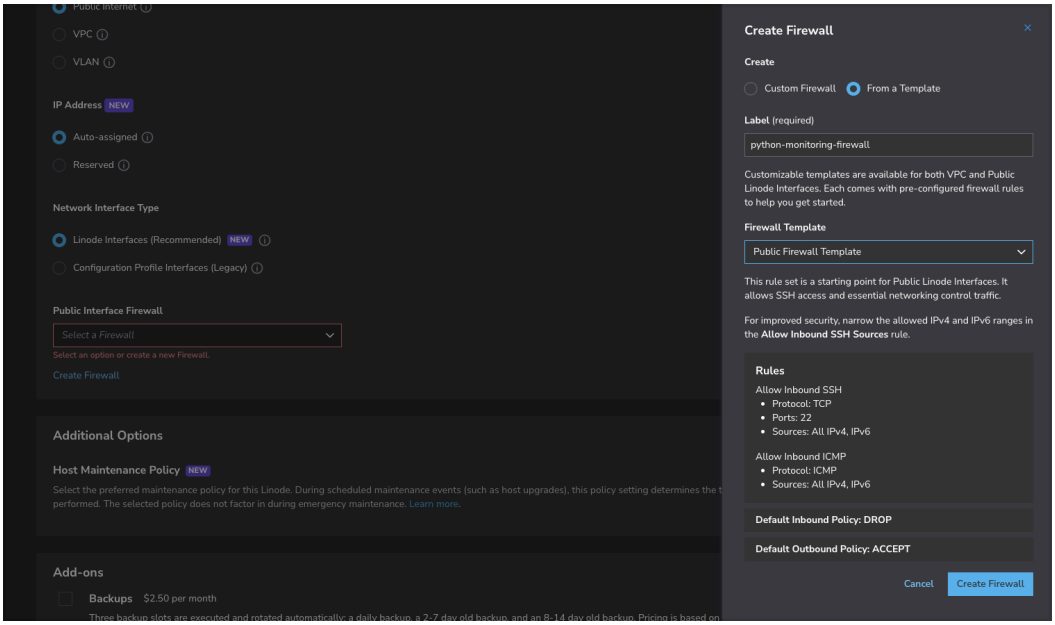

```
❯ cd ~/.ssh
❯ ls
config                id_ed25519
docker-server.pem     id_ed25519.pub
❯ cat id_ed25519.pub
```

And paste it

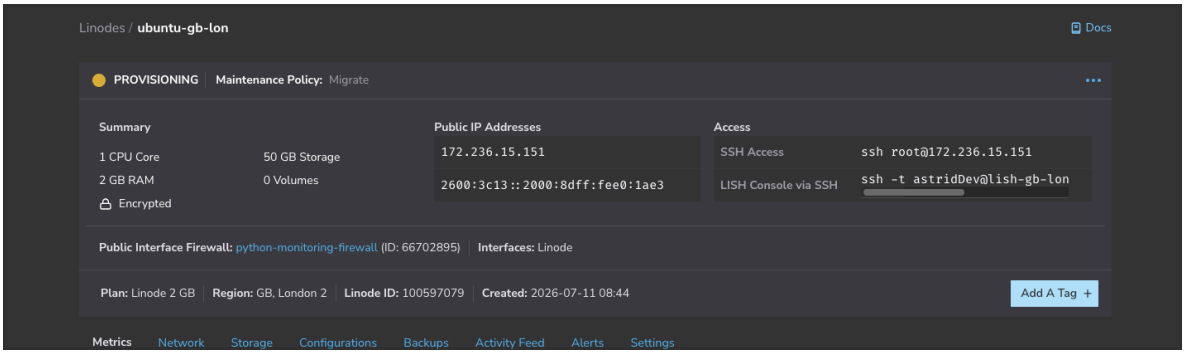




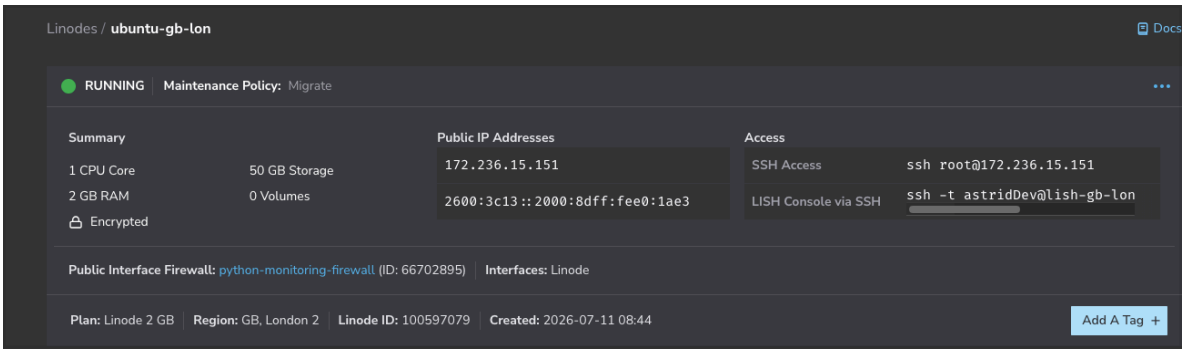
I also added a firewall just in case:



And then I create the server



Now is ready



So I take the ssh info and I access via the terminal
-> ssh root@172.236.15.151

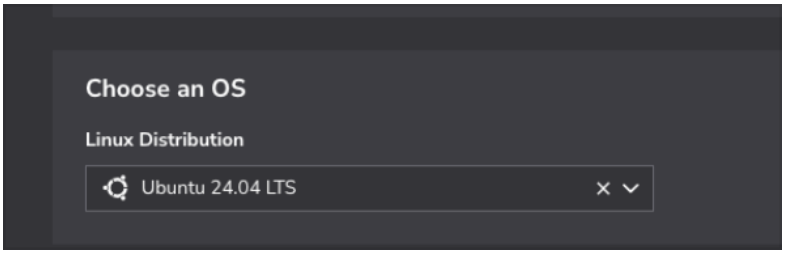
And it asks for the root password so I enter it.

```
❯ ssh root@172.236.15.151
The authenticity of host '172.236.15.151 (172.236.15.151)' can't be established.
ED25519 key fingerprint is SHA256:anO3sIvdHBn/E1Juki4UjrIErj4evNj4V43SZj9hLU4.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.236.15.151' (ED25519) to the list of known hosts.
root@172.236.15.151's password:
```

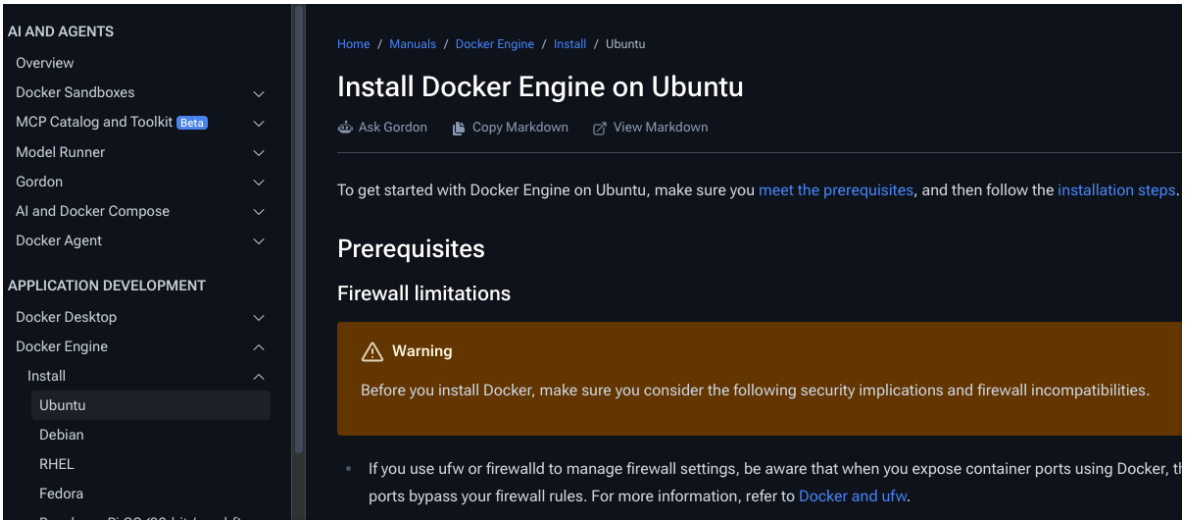
And I am In!

```
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Jul 11 08:48:27 AM UTC 2026

  System load:           0.42
  Usage of /:            4.8% of 48.63GB
  Memory usage:          7%
  Swap usage:            0%
  Processes:             107
  Users logged in:       0
  IPv4 address for eth0: 172.236.15.151
  IPv6 address for eth0: 2600:3c13::2000:8dff:fee0:1ae3

The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@localhost:~#
```

We now install docker for ubuntu as our server uses that OS



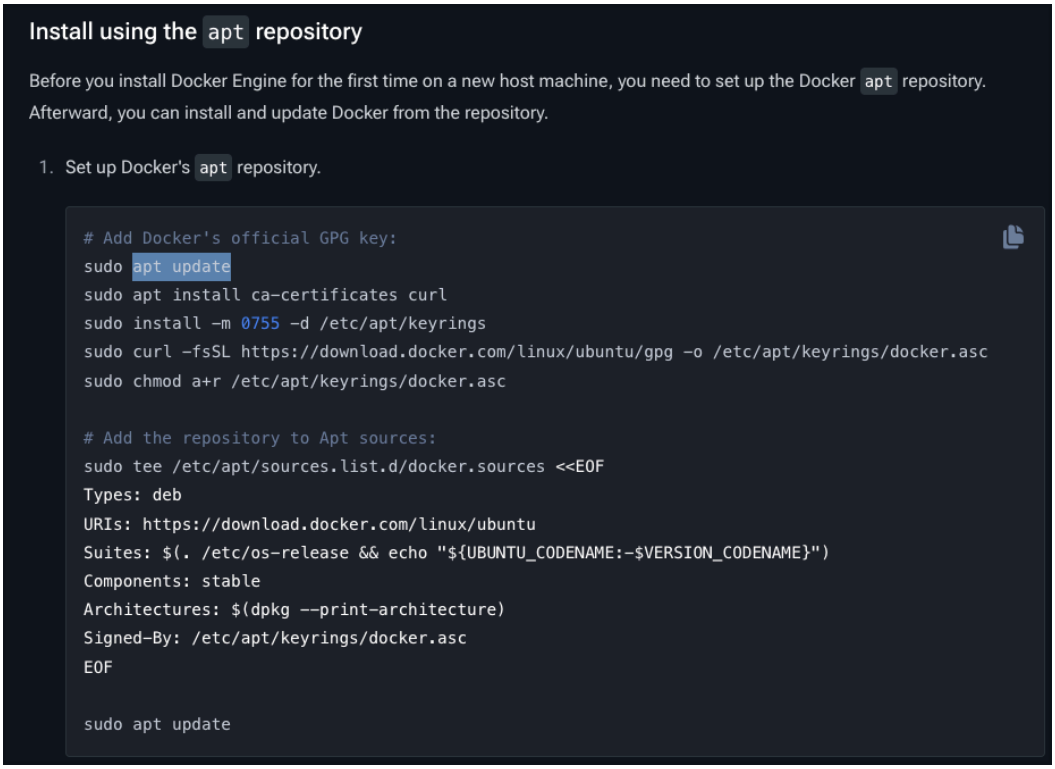
So we follow this documentation https://docs.docker.com/engine/install/ubuntu/



And we can double check:
-> cat /etc/os-release

```
root@localhost:~# cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
root@localhost:~#
```

And because we are logged in the server using the root user we don’t need to use ‘sudo’



Install using the `apt` repository

Before you install Docker Engine for the first time on a new host machine, you need to set up the Docker `apt` repository. Afterward, you can install and update Docker from the repository.

1. Set up Docker's `apt` repository.

```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

-> apt update

```
root@localhost:~# apt update
Get:1 http://mirrors.linode.com/ubuntu noble InRelease
Get:2 http://mirrors.linode.com/ubuntu noble-updates InRelease
Get:3 http://mirrors.linode.com/ubuntu noble-backports InRelease
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126
Get:5 http://mirrors.linode.com/ubuntu noble/main amd64 Packages [1,4
```

Nowlet's make sure we can validate the official Docker repository by adding Docker's official keys:

-> apt install ca-certificates curl

```
root@localhost:~# apt install ca-certificates curl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

-> install -m 0755 -d /etc/apt/keyrings

```
root@localhost:~# install -m 0755 -d /etc/apt/keyrings
root@localhost:~#
```

And now we can verify the key with this command:

-> curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/

keyrings/docker.asc

-> chmod a+r /etc/apt/keyrings/docker.asc

```
root@localhost:~# curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
root@localhost:~# chmod a+r /etc/apt/keyrings/docker.asc
root@localhost:~#
```

Now we set up a stable repository and run apt update to make sure that we have access to the latest version

-> tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

```
root@localhost:~# tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
root@localhost:~#
```

-> apt update

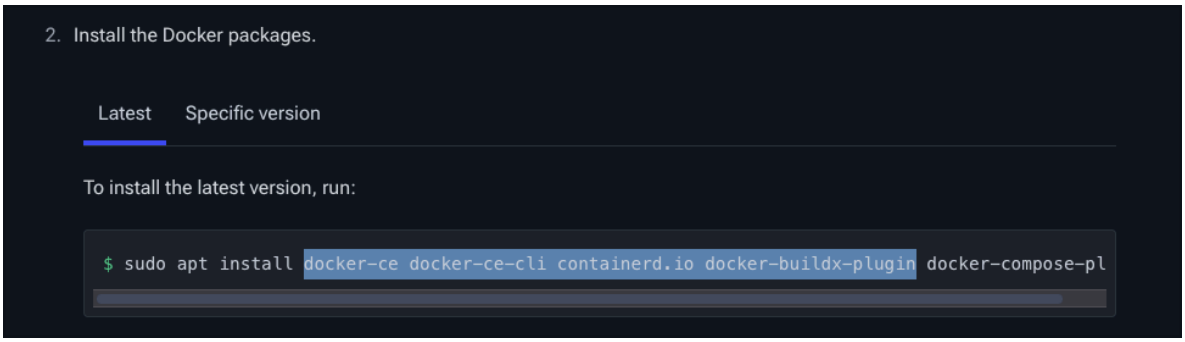
```
root@localhost:~# apt update
Hit:1 http://mirrors.linode.com/ubuntu noble InRelease
Hit:2 http://mirrors.linode.com/ubuntu noble-updates InRelease
Hit:3 http://mirrors.linode.com/ubuntu noble-backports InRelease
Get:4 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:5 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [60.5 kB]
Hit:6 http://security.ubuntu.com/ubuntu noble-security InRelease
Fetched 109 kB in 4s (29.6 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
87 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@localhost:~#
```

Now that we have the repository set up, we can install docker from that

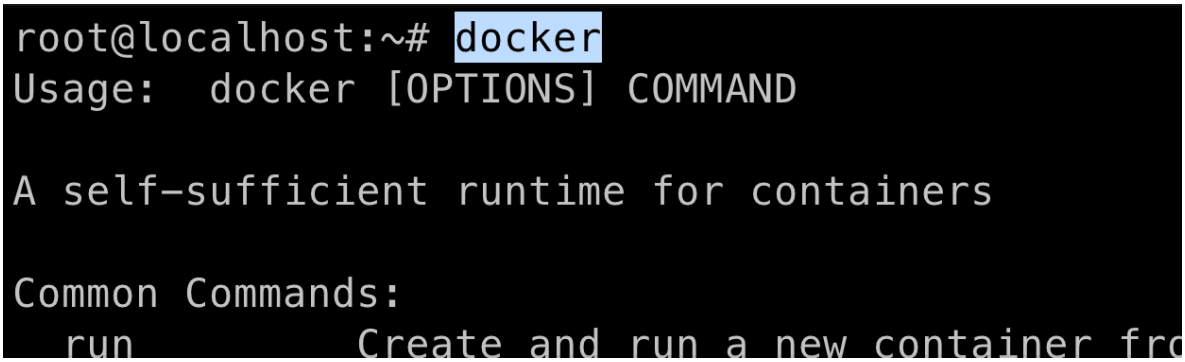
repository.

-> apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

We will install Docker, Docker CE, CLI. Etc



So now if we run
-> docker



It works, docker is installed!

Next is to start an Nginx container in this Linode server

-> docker run -d -p 8080:80 nginx

```
root@localhost:~# docker run -d -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
1645c1e06f46: Pull complete
e95a6c7ea7d4: Pull complete
df68ee7e7a00: Pull complete
1b30016634d5: Pull complete
acf093e7a04f: Pull complete
cd9307c9ecd8: Pull complete
fcb6fd84b2a0: Pull complete
e2c07e54e55a: Download complete
1cf7d051b485: Download complete
Digest: sha256:ec4ed8b5299e5e90694af7750eb6dffd2627317d30544d056b0371f8082f7bce
Status: Downloaded newer image for nginx:latest
52527a5428e85ce58f8ae7829f6f9018bd778f5b6fcc4f08ff4812541f902eb9
root@localhost:~#
```

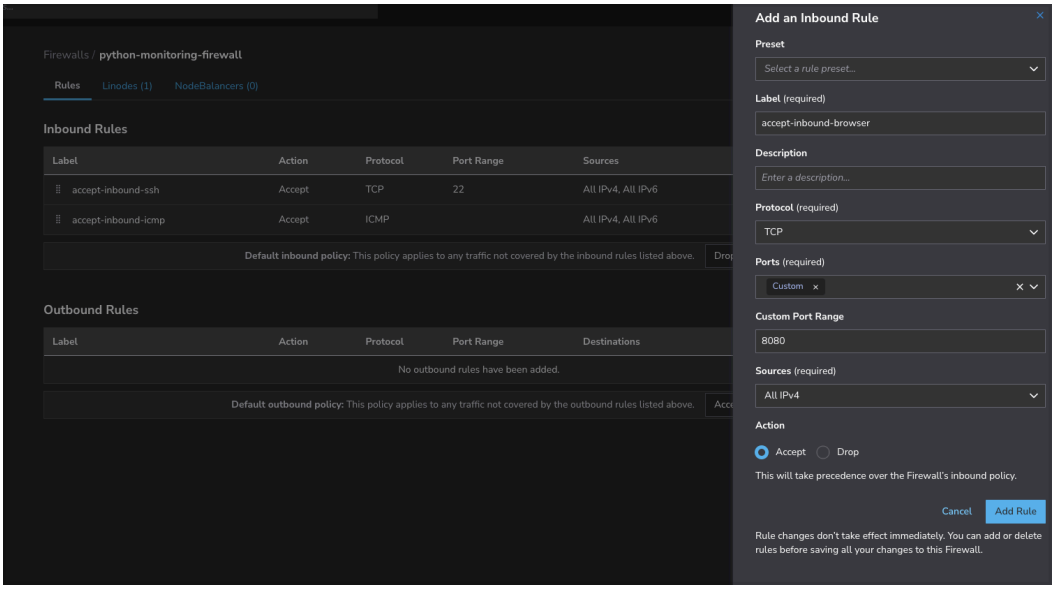
And the container is running now:

-> docker ps

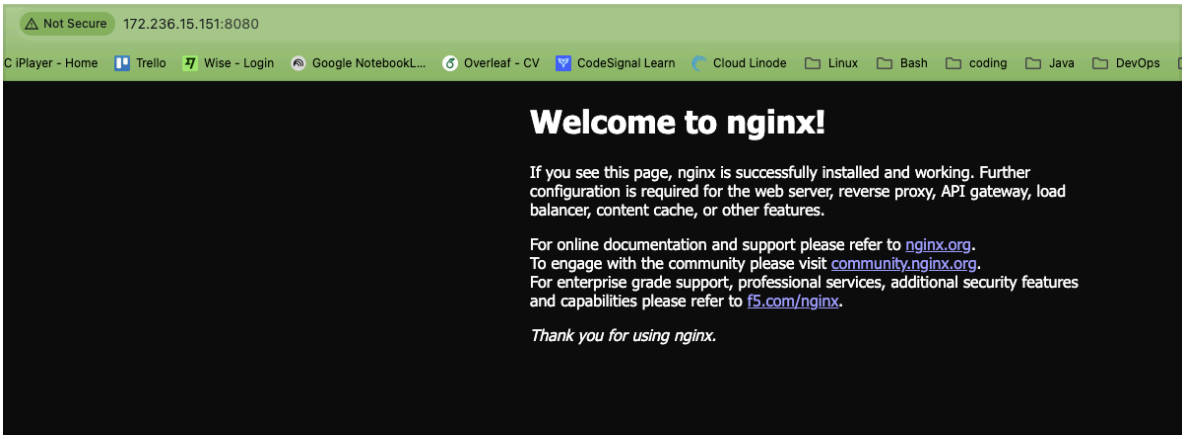
```
root@localhost:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS
               NAMES
52527a5428e8   nginx     "/docker-entrypoint.…"   35 seconds ago   Up 35 seconds   0.0.0.0:8080->80/tcp, [::]:
8080->80/tcp   great_elbakyan
root@localhost:~#
```

Now nginx app is running in our Linode server, now let's access it:
-> go to the browser and use the public IP address of the Linode server along with the port
-> 172.236.15.151:8080

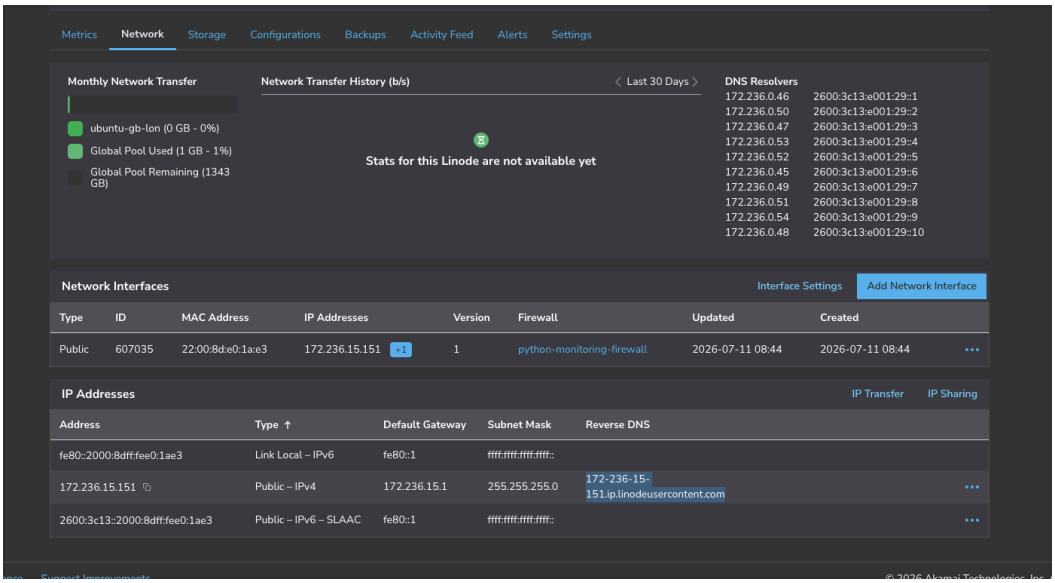
Page didn't load, so I added a new inbound rule for port 8080:



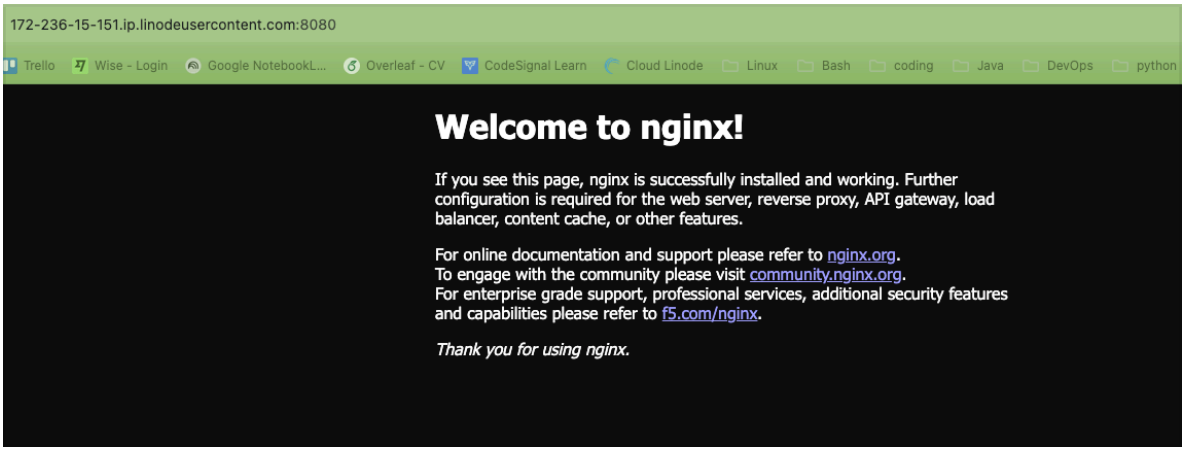
And I am in:



In addition to the IP address we also have the DNS address
-> 172-236-15-151.ip.linodeusercontent.com



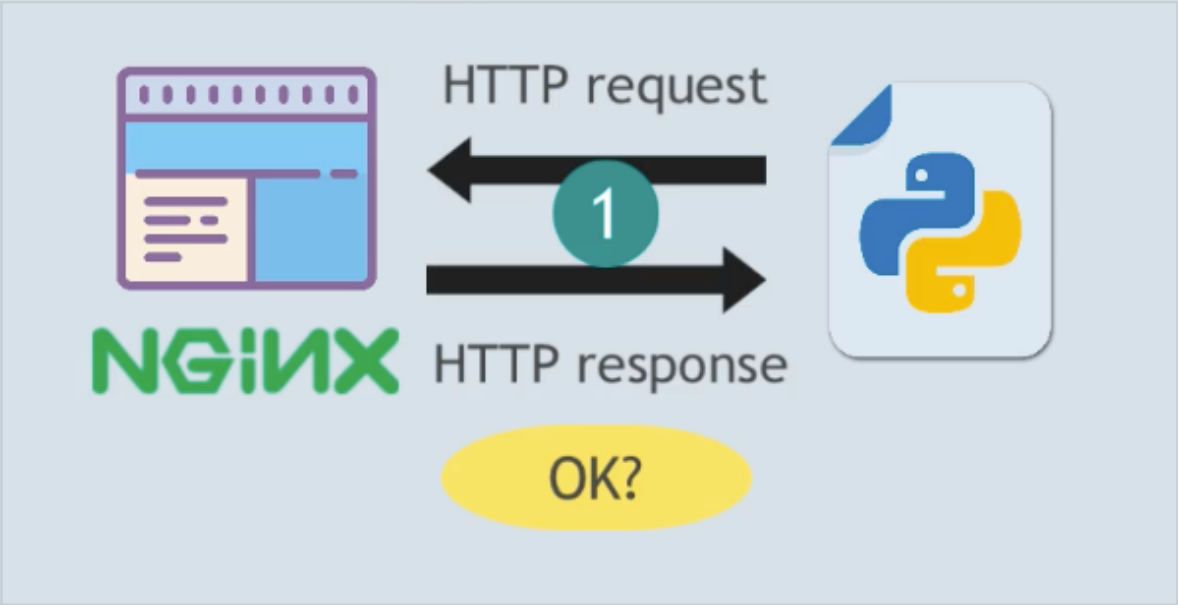
So if we use the DNS instead of the public IP address we can also access the nginx app via the browser:
-> http://172-236-15-151.ip.linodeusercontent.com:8080



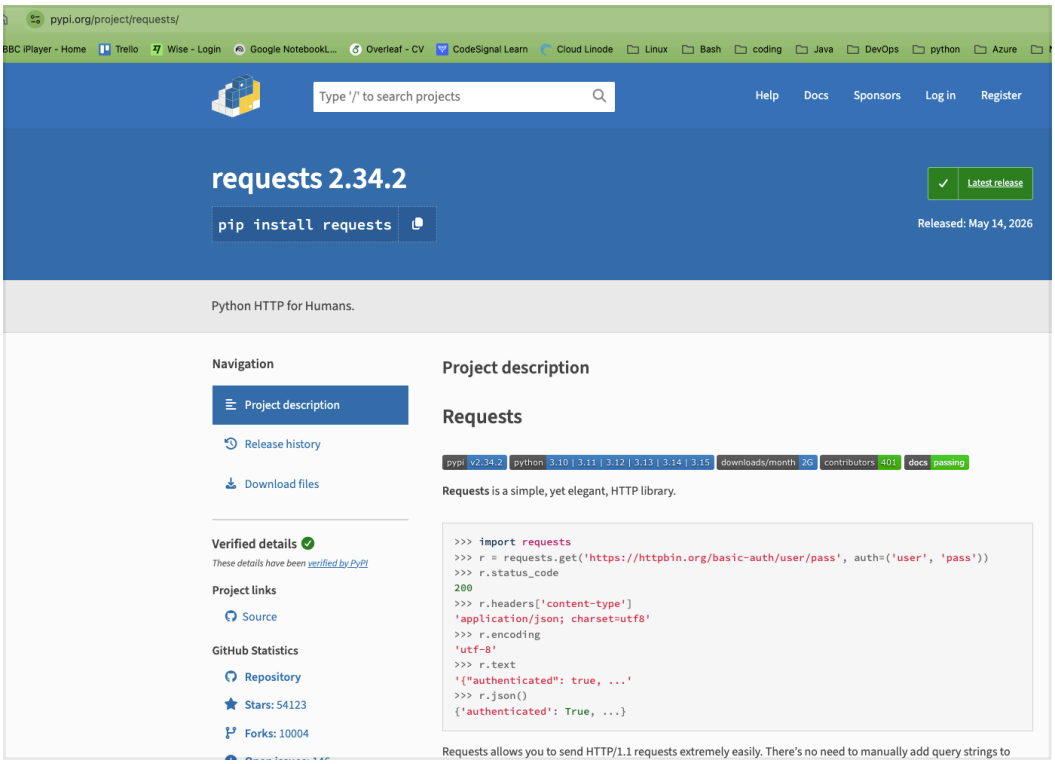
Now let's write our DEMO automation Script in python.

Website Request

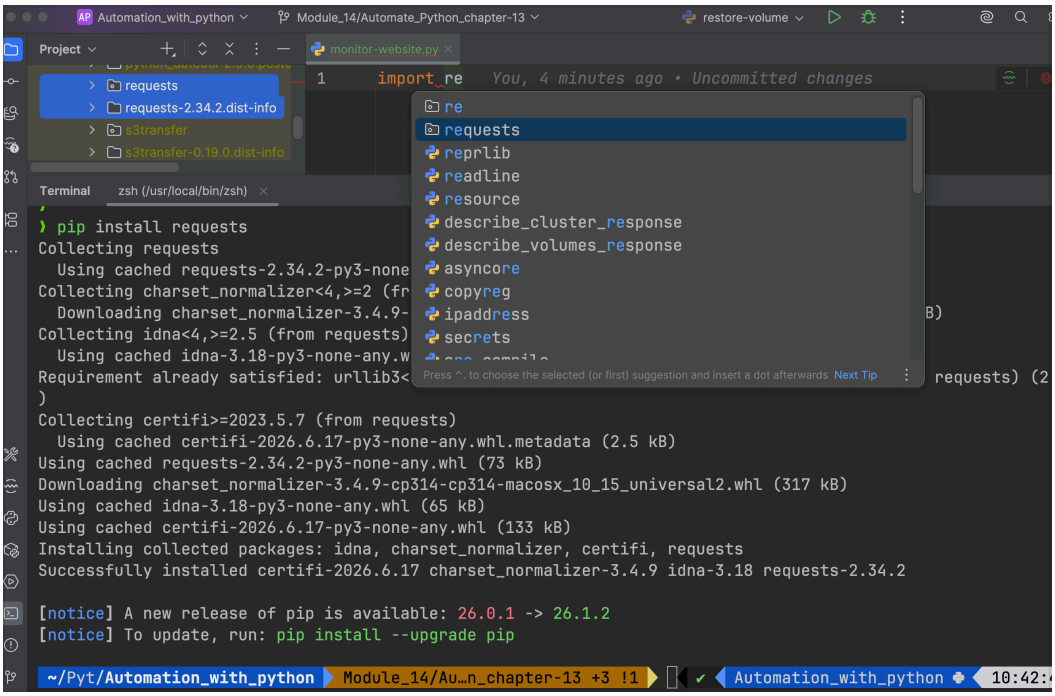
First we will access the browser like we did above but this time using a python program.



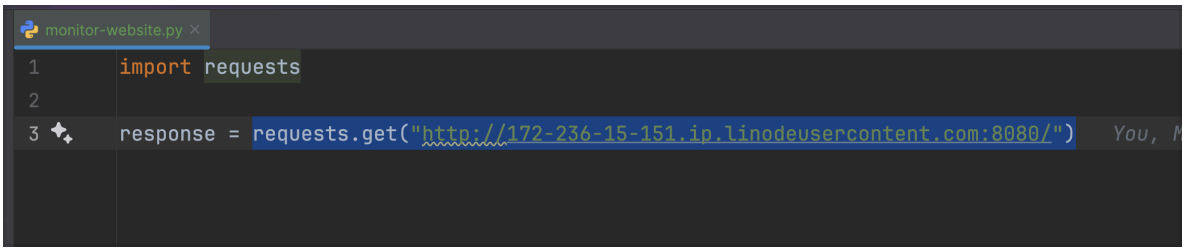
And if we want to talk to other applications from python we can use a library called -> Requests



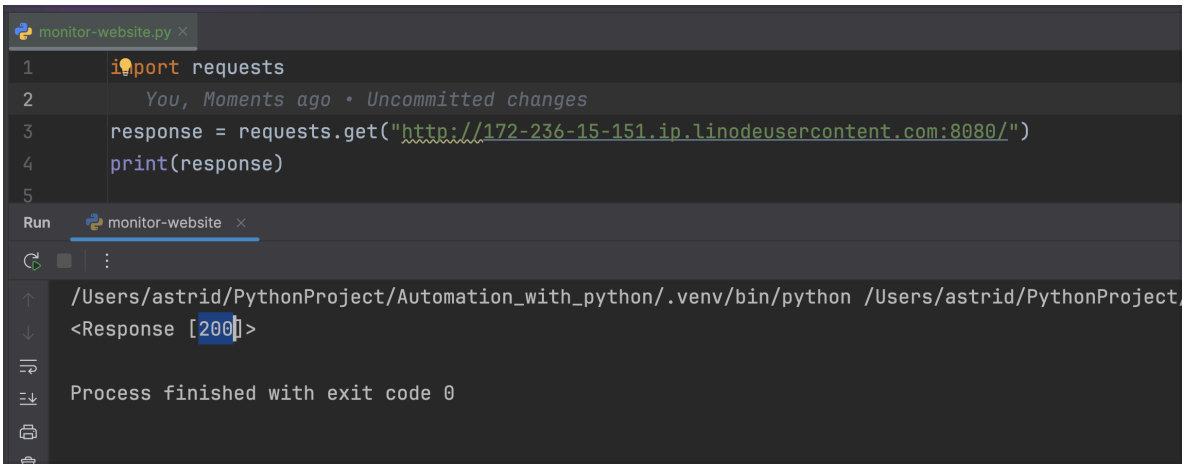
-> pip install requests



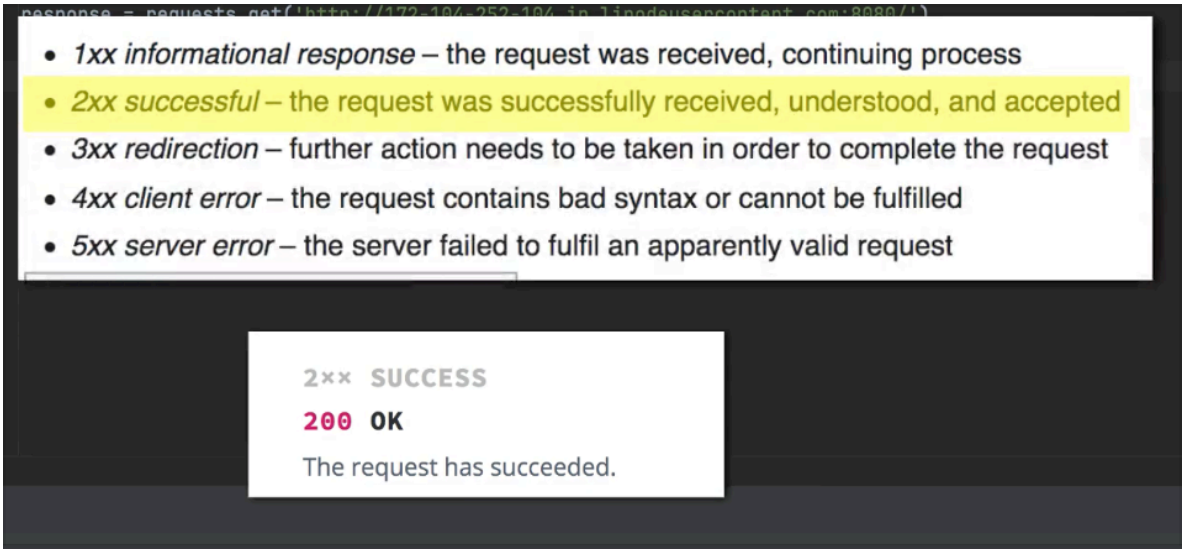
And now we can make the call which is equivalent to provide the dns:8080 in the browser
-> requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")



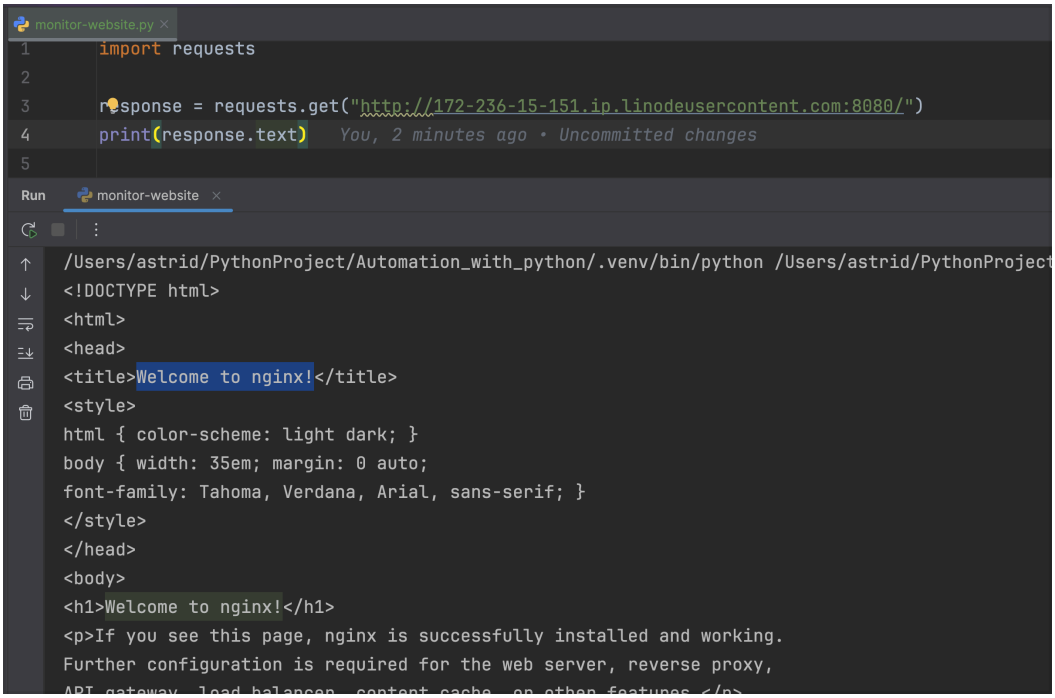
And we can check if the response is successful



And it is, we get 200!

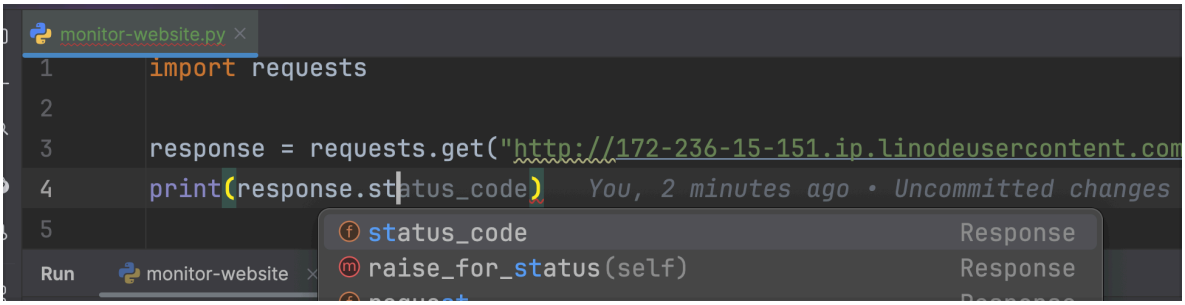


And if we want to see the exact response we see in the browser then we use -> response.text

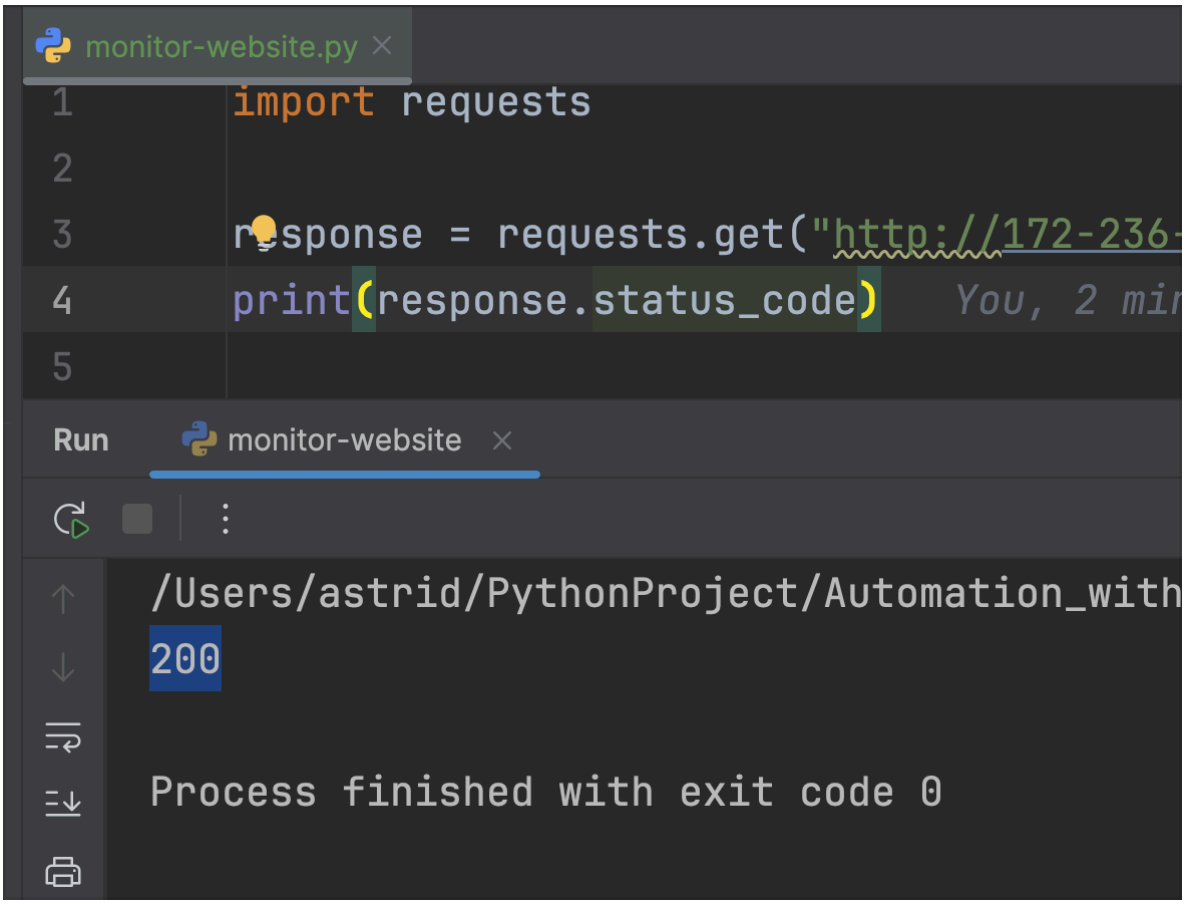


And we want to check if the application is accessible so we want to check if the status code is 200 and the response has an attribute called ‘status_code’

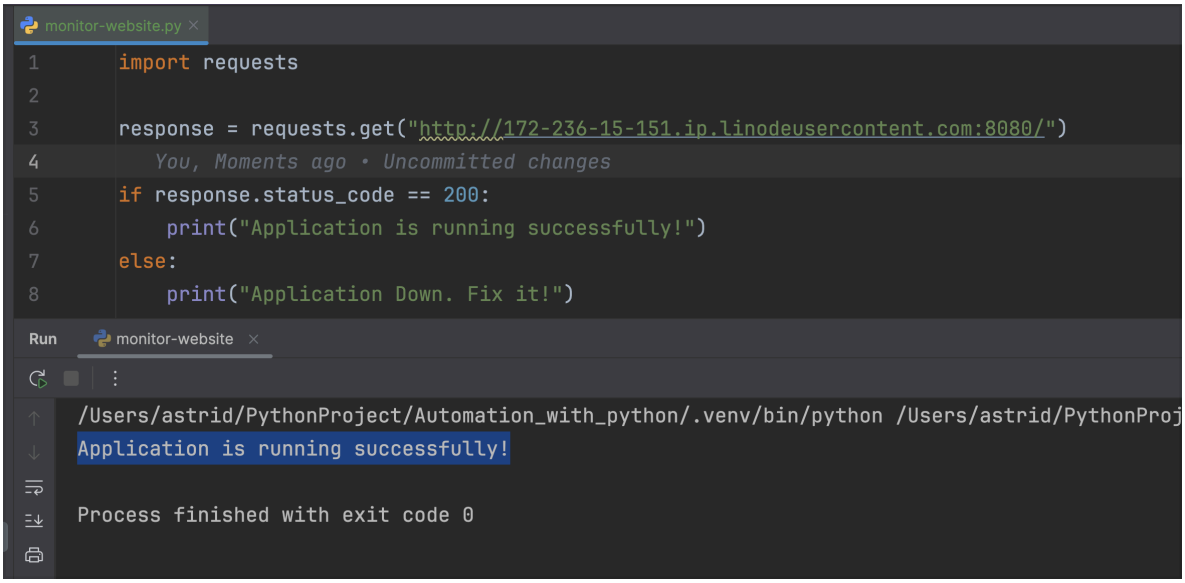
-> response.status_code



So if we use it we can see we get the status code:



So we add logic to print whether the app is accessible or not in a way the user can easily understand and appropriate for monitoring:


monitor-website.py
import requests
response = requests.get("http://172-236-15-151.ip.linodeusercontent.com:8080/")
You, Moments ago • Uncommitted changes
if response.status_code == 200:
print("Application is running successfully!")
else:
print("Application Down. Fix it!")
Run
monitor-website
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProj
Application is running successfully!
Process finished with exit code 0